INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

APLICACIONES PARA COMUNICACIONES EN RED

Reporte

“Servidor HTTP”

Grupo: 6CM2

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Moreno Cervantes Axel Ernesto

Fecha de entrega: 05 de diciembre de 2025

ÍNDICE

INTRODUCCIÓN..... 2
DESARROLLO..... 4
CONCLUSIÓN..... 11

INTRODUCCIÓN

- ## Tipos MIME (Multipurpose Internet Mail Extensions)

Es una estandarización que describe el tipo de contenido que se está enviando o recibiendo en internet y su propósito es indicar al sistema cómo debe interpretar el contenido (texto, imagen, audio, documento, etc.)

¿Por qué son importantes?

Porque permiten que:

- ✓ Los navegadores sepan cómo abrir un archivo.
- ✓ Los servidores web devuelvan contenido correcto.
- ✓ Las APIs intercambien datos de forma estandarizada.
- ✓ Los clientes de correo muestren mensajes y adjuntos.
- ✓ Se eviten errores de interpretación o seguridad.

Tipo principal	Ejemplos	Propósito
text/	html, css, plain	Contenido textual
application/	json, xml, pdf	Datos estructurados o binarios
image/	png, jpg, gif	Imágenes
audio/	mp3, ogg	Audio
video/	mp4, webm	Video
multipart/	form-data	Combinaciones de contenido

- ## Peticiones HTTP

Una petición HTTP (HTTP request) es un mensaje enviado por un cliente (navegador, app móvil o script) hacia un servidor, con el objetivo de solicitar un recurso o ejecutar una operación.

Una petición HTTP está compuesta por:

1. **Línea de petición (Request Line)**
   - Incluye: Método / Ruta or URL / Versión del protocolo
2. **Cabeceras (Headers)**
   - Información adicional que describe la petición.
3. **Cuerpo (Body)**
   - Sirve para enviar datos, por ejemplo, en formularios o APIs (opcional)

Principales métodos HTTP

Los métodos indican qué acción se quiere realizar.

- **GET:** Solicita un recurso (no tiene cuerpo).
- **POST:** Crea un recurso nuevo o envía datos.
- **PUT**: Remplaza completamente un recurso existente.
- **DELETE:** Elimina un recurso.
- **PATCH:** Modifica parcialmente un recurso.
- **HEAD:** Es como GET, no tiene cuerpo, pero se usa para obtener metadatos.
- **OPTIONS:** Pregunta qué métodos soporta un recurso (importante para CORS).

• Códigos de respuesta HTTP

Los códigos de respuesta HTTP son números de tres dígitos que un servidor devuelve al cliente (navegador) para indicar el resultado de una petición HTTP, por ejemplo, si la solicitud fue exitosa, hubo un error, el recurso no existe, si se requiere una acción adicional, si hay problemas del lado del servidor, etc.

Clasificación oficial de los códigos HTTP

Cada código pertenece a una clase, identificada por su primer dígito.

Rango	Nombre	Significado
1xx	Informativos	El servidor recibió la solicitud; continúa el proceso
2xx	Éxito	La solicitud fue recibida y procesada correctamente
3xx	Redirecciones	Se necesita otra acción para completar la solicitud
4xx	Error del cliente	La solicitud tiene un error; no puede procesarse
5xx	Error del servidor	El servidor falló al procesar una solicitud válida

DESARROLLO

Para el servidor HTTP solo se creó un solo archivo.

- Variables “globales”

```
HOST = "127.0.0.1"

PUERTO_PRIMARIO = 8080
PUERTO_SECUNDARIO = 8081

CARPETA_SCRIPT = Path(__file__).parent
CARPETA_PRIMARIA = CARPETA_SCRIPT / "archivos1"
CARPETA_SECUNDARIA = CARPETA_SCRIPT / "archivos2"

POOL_MAX = 6
MITAD_POOL = POOL_MAX // 2
pool = threading.Semaphore(POOL_MAX)

conexiones_activas = 0
lock = threading.Lock()

segundo_servidor_iniciado = False

MIME_TYPES = {
    ".html": "text/html",
    ".json": "application/json",
    ".txt": "text/plain",
    ".png": "image/png",
    ".jpg": "image/jpeg",
    ".jpeg": "image/jpeg",
    ".mp4": "video/mp4",
    ".mp3": "audio/mpeg",
    ".pdf": "application/pdf",
}
```

Imagen 1. Variables globales.

Aquí se declaran las variables necesarias para el funcionamiento del programa. Se define la dirección IP del servidor y los puertos para ambos servidores. También se obtiene la ruta de las carpetas donde se alojan los recursos para ambos servidores.

Se declaran las variables para el pool de conexiones, ‘POO_MAX’ y ‘MITAD_POOL’ para obtener la mitad del máximo de conexiones para activar el segundo servidor. El contador para las conexiones activas en el servidor, la bandera para activar el segundo servidor y por último un diccionario con los tipos de MIME.

- **ahora()**

```
def ahora() -> str:
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

Imagen 2. Función ahora().

Descripción general: Esta función se encarga de devolver la hora actual como una cadena y de esta manera mostrar la hora en la que se envía una petición al servidor.

- **obtener_mime(archivo: Path)**

```python
def obtener_mime(archivo: Path) -> str:
    return MIME_TYPES.get(archivo.suffix, "application/octet-stream")
```

Imagen 2. Función obtener_mime().

Descripción general: Obtiene la extensión del recurso de la petición para buscarla en el diccionario 'MIME_TYPES' y devolver el MIME correspondiente como cadena.

Parámetros:

- **archivo : Path**
  - Nombre del recurso solicitado en la petición al servidor.
- **leer_body()**

```python
def leer_body(peticion: str) -> str:
    separador = "\r\n\r\n"

    if separador in peticion:
        return peticion.split(separador, 1)[1]

    return ""
```

Imagen 3. Función leer_body(peticion: str).

Descripción general: Obtiene el cuerpo de la petición enviada al servidor, separándolo por medio de la cadena '\r\n\r\n' del encabezado, para al final devolverlo como una cadena, pero si la petición no tiene cuerpo entonces devuelve una cadena vacía.

Parámetros:

- peticion : str
  - Petición enviada al servidor.
- **enviar_respuesta(cliente: socket.socket, status_line: str, content_type: str, body: str)**

```python
def enviar_respuesta(cliente: socket.socket, status_line: str, content_type: str, body: str) -> None:
    body_bytes = body.encode("utf-8")

    cabecera = (
        f"{status_line}\r\n"
        f"Content-Type: {content_type}\r\n"
        f"Content-Length: {len(body_bytes)}\r\n"
        f"\r\n"
    )
    cliente.sendall(cabecera.encode() + body_bytes)
```

Imagen 4. Función enviar_respuesta().

Descripción general: Construye la cabecera de la respuesta a la petición, incluyendo el estatus de la petición, el contenido de la respuesta (recurso pedido) y el tamaño del recurso pedido, para al final enviar la respuesta al navegador incluyendo el cuerpo y la cabecera.

Parámetros:

- **cliente : socket.socket**
  - Socket del cliente al quien se envía la respuesta.
- **Status_line : str**
  - Estatus de la respuesta.
- **content_type : str**
  - Tipo del contenido de acuerdo con el MIME.
- **body : str**
  - Contenido de la respuesta.
- **procesar_peticion(cliente: socket.socket, peticion: str, carpeta_base: Path, puerto: int)**

```python
def procesar_peticion(cliente: socket.socket, peticion: str, carpeta_base: Path, puerto: int) -> None:
    linea = peticion.splitlines()[0]
    metodo, ruta, _ = linea.split()

    if ruta == "/":
        ruta = "/index.html"

    archivo = carpeta_base / ruta.lstrip("/")
```

Imagen 5. Función procesar_peticion().

Descripción general: De la petición, se extrae el método y la ruta del recurso pedido. De acuerdo con el método, se construye la cabecera correspondiente para la respuesta a la petición y al final se envía la respuesta al navegador con el código de respuesta y el MIME correspondientes.

Parámetros:

- **cliente : socket.socket**
  - Socket del cliente a quien se envía la respuesta.
- **peticion : str**
  - Texto bruto recibido (headers + body).
- **carpeta_base : Path**
  - Carpeta con los archivos solicitados en la petición.
- **puerto : int**
  - Puerto del servidor.

En este bloque de código se obtiene el método y la ruta del recurso pedido y se maneja la redirección a index.html.

```python
    # ====== GET ======
    if metodo == "GET":
        if archivo.exists() and archivo.is_file():
            contenido = archivo.read_bytes()
            mime = obtener_mime(archivo)

            cabecera = (
                "HTTP/1.1 200 OK\r\n"
                f"Content-Type: {mime}\r\n"
                f"Content-Length: {len(contenido)}\r\n"
                "\r\n"
            ).encode()

            print("_" * 50)
            print(f"{utils.BLUE}<====== RESPUESTA ({puerto}) ======>{utils.RESET}")
            print(cabecera.decode(errors="ignore"))
            print("_" * 50)

            cliente.sendall(cabecera + contenido)
        else:
            body = "<h1>404 - Archivo no encontrado</h1>"
            enviar_respuesta(cliente, "HTTP/1.1 404 Not Found", "text/html", body)
```

Imagen 6. Procesar petición GET.

```python
    # ====== POST ======
    elif metodo == "POST":
        body = leer_body(peticion)

        archivo.write_text(body, encoding="utf-8")
        respuesta_body = f"Archivo {archivo.name} creado correctamente en {carpeta_base}"

        enviar_respuesta(
            cliente,
            "HTTP/1.1 201 Created",
            "text/plain",
            respuesta_body
        )
```

Imagen 7. Procesar petición POST.

```python
    # ====== PUT ======
    elif metodo == "PUT":
        body = leer_body(peticion)

        if archivo.exists():
            archivo.write_text(body, encoding="utf-8")

            respuesta_body = f"Archivo {archivo.name} actualizado correctamente"

            enviar_respuesta(
                cliente,
                "HTTP/1.1 200 OK",
                "text/plain",
                respuesta_body
            )
        else:
            enviar_respuesta(
                cliente,
                "HTTP/1.1 404 Not Found",
                "text/plain",
                "Archivo no existe"
            )
```

Imagen 8. Procesar petición PUT.

Para este método también se considera el caso en el que el archivo que se quiere modificar no existe, para ello se envía la respuesta correspondiente.

```
138     # ====== DELETE ======
139     elif metodo == "DELETE":
140         if archivo.exists():
141             archivo.unlink()
142
143             respuesta_body = f"Archivo {archivo.name} eliminado"
144
145             enviar_respuesta(
146                 cliente,
147                 "HTTP/1.1 200 OK",
148                 "text/plain",
149                 respuesta_body
150             )
151         else:
152             enviar_respuesta(
153                 cliente,
154                 "HTTP/1.1 404 Not Found",
155                 "text/plain",
156                 "Archivo no encontrado"
157             )
```

Imagen 9. Procesar petición DELETE.

Para este método también se considera el caso en el que el archivo que se quiere eliminar no existe, para ello se envía la respuesta correspondiente.

```
159     # ====== OTROS ======
160     else:
161         enviar_respuesta(
162             cliente,
163             "HTTP/1.1 405 Method Not Allowed",
164             "text/plain",
165             "Método no permitido"
166         )
```

Imagen 10. Procesar métodos no soportados por el servidor.

- **atender_cliente(cliente: socket.socket, carpeta_base: Path, puerto: int)**

```
169 def atender_cliente(cliente: socket.socket, carpeta_base: Path, puerto: int) -> None:
170     if not pool.acquire(blocking=False):
171         cliente.sendall(
172             b"HTTP/1.1 503 Service Unavailable\r\n"
173             b"Content-Type: text/plain\r\n\r\n"
174             b"Servidor saturado"
175         )
176         cliente.close()
177
178         return
```

Imagen 11. Función antender_cliente().

Descripción general: Se encarga de atender al cliente, es decir, se encarga de recibir las peticiones, actualizar el número de conexiones activas, cambiar de servidor cuando se supera la mitad del pool de conexiones

Parámetros:

- **cliente : socket.socket**
  - Socket del cliente al quien se envía la respuesta.
- **carpeta_base : Path**
  - Carpeta con los archivos solicitados en la petición.
- **puerto : int**
  - Puerto del servidor.

En este bloque de código se maneja la saturación del servidor.

```
    global conexiones_activas

    conexion_id = uuid.uuid4().hex[:8]

    with lock:
        conexiones_activas += 1
        print(f"[{ahora()}] [+] [ID: {conexion_id}] ({puerto}) Conexiones activas: {conexiones_activas}")

    try:
        datos = cliente.recv(4096)
        peticion = datos.decode("utf-8", errors="ignore")

        if not peticion:
            return

        linea = peticion.splitlines()[0]
        print(f"[{ahora()}] [ID: {conexion_id}] PETICIÓN => {linea}")

        with lock:
            redirigir = conexiones_activas > MITAD_POOL and puerto == PUERTO_PRIMARIO

        if redirigir:
            respuesta = (
                "HTTP/1.1 302 Found\r\n"
                f"Location: http://{HOST}:{PUERTO_SECUNDARIO}\r\n\r\n"
            )

            cliente.sendall(respuesta.encode())
            print(f"[{ahora()}] [ID: {conexion_id}] REDIRECCIONANDO A PUERTO {PUERTO_SECUNDARIO}")

            return

        procesar_peticion(cliente, peticion, carpeta_base, puerto)
```

Imagen 12. Recibir la petición, actualización de las conexiones activas, redirección al puerto secundario.

```python
        print(f"[{ahora()}] [ID: {conexion_id}] RESPUESTA enviada correctamente")
        time.sleep(2)
    except Exception as e:
        print(f"[{ahora()}] [ID: {conexion_id}] ERROR: {e}")

        try:
            cliente.sendall(
                b"HTTP/1.1 500 Internar Server Error\r\n"
                b"Content-Type: text/plain\r\n\r\n"
                b"Error interno del servidor"
            )
        except:
            pass

    finally:
        cliente.close()

        with lock:
            conexiones_activas -= 1
            print(f"[{ahora()}] [-] [ID: {conexion_id}] ({puerto}) Conexiones activas: {conexiones_activas}")

        pool.release()
```

Imagen 13. Manejo de la excepción al recibir la petición.

- **iniciar_servidor(pierto: int, carpeta_base: Path)**

```python
def iniciar_servidor(puerto: int, carpeta_base: Path) -> None:
    global segundo_servidor_iniciado

    servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    servidor.bind((HOST, puerto))
    servidor.listen()

    print(f"Servidor escuchando en http://{HOST}:{puerto} usando {carpeta_base}")

    while True:
        cliente, _ = servidor.accept()

        iniciar_segundo = False

        with lock:
            if conexiones_activas >= MITAD_POOL and not segundo_servidor_iniciado and puerto == PUERTO_PRIMARIO:
                segundo_servidor_iniciado = True
                iniciar_segundo = True

        if iniciar_segundo:
            print(f"\n{utils.RED}>>> INICIANDO SEGUNDO SERVIDOR (8081) <<<{utils.RESET}\n")

            threading.Thread(
                target=iniciar_servidor,
                args=(PUERTO_SECUNDARIO, CARPETA_SECUNDARIA),
                daemon=True
            ).start()

        hilo = threading.Thread(
            target=atender_cliente,
            args=(cliente, carpeta_base, puerto)
        )
        hilo.start()

if __name__ == "__main__":
    iniciar_servidor(PUERTO_PRIMARIO, CARPETA_PRIMARIA)
```

Imagen 14. Creación e inicio del servidor.

Descripción general: Se encarga de crear el servidor, de aceptar conexiones y de iniciar al segundo servidor al sobrepasar la mitad del pool de conexiones.

Parámetros:

- **puerto : int**
  - Puerto del servidor.
- **carpeta_base : Path**
  - Carpeta con los recursos solicitados en la petición.

CONCLUSIÓN

- **García Escamilla Bryan Alexis**

Esta práctica fue clave para entender cómo es que funciona un servidor HTTP internamente. Para ello el cliente envía una petición al servidor, este acepta la conexión y la recibe. Lo primero que verifica es el encabezado de donde obtiene el método (en este caso GET, POST, PUT y DELETE) y posteriormente el nombre del recurso, entonces internamente el servidor procesa la petición de acuerdo con el método, para el caso de GET verifica que el recurso que se quiere ver exista, verifica el tipo de archivo, lo busca y envía la respuesta indicando el tipo y tamaño del recurso, de lo contrario la respuesta va a ser el código '404' indicando que no se encontró el recurso, para POST solo se obtiene el nombre del recurso que se quiere crear y el servidor envía la misma respuesta que con GET pero indicando que el recurso fue creado, para PUT resulta ser muy similar que GET solo que en caso de que exista el recurso se envía como respuesta la confirmación de que el recurso fue modificado y en el caso del DELETE lo mismo pero para indicar que fue eliminado.

Para el pool de conexiones, se verifica que haya la mitad de las conexiones activas y una vez que esto se cumple se inicia al segundo servidor en otro puerto, y entonces las peticiones siguientes serán redirigidas a este segundo servidor. En nuestro caso el servidor se inicia solo una vez (cuando se cumple la condición por primera vez) y se muestra el mensaje correspondiente, las demás veces solo se redirige sin indicarlo, la única manera es observar en la respuesta del servidor y en la actualización de las conexiones activas el puerto desde donde se responde y el puerto de la conexión activa.

- **Meléndez Macedonio Rodrigo**

Realizar esta practica fue fundamental, ya que durante el desarrollo de esta entendí como se comporta un servidor HTTP al momento de que un usuario como yo interactúa con este. Primero, el servidor debe identificar que es lo que acción quiere realizar el cliente, para ello se utiliza un encabezado en el que se debe de especificar el tipo de método que se va a utilizar (GET, POST, PUT, DELETE, entre otros). Cada método tiene una diferente funcionalidad y se utilizan en diferentes contextos; por ejemplo, en nuestro caso utilizamos el método GET dentro de la petición para que el cliente obtenga como respuesta diferentes archivos especificados con los tipos MIME (html, json, txt, etc), sin embargo, en caso de que el archivo no exista o no se haya creado mediante POST, el servidor enviará un código de respuesta HTTP (404) para avisar al cliente que es lo que ocurrió. Por otro lado, el método POST lo utilizamos para crear un archivo de tipo MIME (txt) a partir de esta petición y enviar como respuesta al cliente el contenido de este archivo. En el caso del método PUT, lo utilizamos para sobrescribir una cadena de texto dentro del archivo (txt) que el cliente creo, aquí, la

cadena la debe de escribir el usuario desde el navegador para enviar la petición al servidor y obtener como respuesta el archivo sobrescrito, así como un mensaje de confirmación. Por último, utilizamos el método DELETE para eliminar el mismo archivo (txt) que creamos con POST, de esta manera el cliente obtiene como respuesta la eliminación del archivo y un mensaje notificando sobre esa acción realizada.